A Staged Framework for Corruption-Aware Image Restoration and Style-Conditioned Sketch Synthesis: From Universal Autoencoders to Mixture-of-Experts Routing and Conditional Adversarial Generation

Muhammad Ismail
Department of Computer Science
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i232524@isb.nu.edu

***Abstract*—We design, train, evaluate and deploy four related generative systems. Tasks 1–3 restore Oxford-IIIT Pet images (128×128) affected by salt-and-pepper noise, Gaussian blur or rectangular occlusion, generated at runtime with deterministic validation/test manifests. Task 1 is a universal convolutional denoising autoencoder with a spatial $128\times8\times8$ bottleneck and an AirNet-style contrastive degradation encoder that FiLM-conditions the network without ever receiving the corruption label; its 60-epoch run reaches validation SSIM 0.78, although the checkpoint available for testing is an early one (test SSIM 0.40). Task 2 trains a four-class corruption classifier on exactly class-balanced batches (test accuracy 98.27%, macro-F1 0.973) that hard-routes inputs to three independently trained specialist autoencoders, with an identity bypass for clean images. Task 3 converts this system into a differentiable soft mixture of experts initialised from Task 2 and jointly fine-tuned with reconstruction, SSIM, classification and load-balancing losses; it improves test SSIM over hard routing on every corrupted condition (up to $+0.106$ on low-severity occlusion; pooled SSIM 0.764). Task 4 is a style-conditioned conditional GAN (U-Net generator with a learned style embedding injected through FiLM, style-conditioned PatchGAN discriminator) trained on FS2K, reaching test SSIM 0.444 overall and 0.620 on Style 3. Every model is tuned with Optuna, tracked with MLflow, exported to ONNX (maximum PyTorch–ONNX deviation $\leq 2.6\times10^{-5}$) and served through a Google-Stitch-designed React/Tailwind front end and a FastAPI back end orchestrated with Docker Compose.**

***Index Terms*—image restoration, denoising autoencoder, mixture of experts, conditional GAN, Optuna, ONNX, MLflow**

Code: https://github.com/MIsmail2524/GenAI_Assignment1
Demo video: https://youtu.be/1w7otgAITqU?si=ZKg175hyqZrZvtf

I. Introduction

Real images are degraded by corruptions whose type and severity are rarely known in advance. A deployable restoration system can (i) learn one representation that handles every corruption, (ii) identify the corruption explicitly and delegate to a specialist, or (iii) blend specialists continuously according to confidence. We implement and compare all three strategies on a shared dataset, split and corruption pipeline (Tasks 1–3), and address a related paired image-to-image problem, style-conditioned face-to-sketch generation, with a conditional GAN (Task 4). All four models are integrated into one browser application. The contributions are:

- A runtime corruption pipeline with exact class balance where required and fully reproducible manifests (Sec. III).
- A blind universal autoencoder with a genuine spatial bottleneck and contrastive degradation conditioning (Sec. V).
- A hard-routed classifier-plus-specialists system evaluated under oracle and predicted routing (Sec. VI).
- A jointly trained soft mixture of experts with routing analysis (Sec. VII).
- A style-conditioned pix2pix-type cGAN (Sec. VIII).
- An ONNX/FastAPI/React/Docker deployment with experiment tracking (Sec. IX).

II. Related Research and Alternatives Investigated

Restoration autoencoders. Denoising autoencoders learn to map corrupted inputs to clean targets through a compressed code [2]. Zhao *et al.* [11] showed that L2 training produces blurrier restorations than L1, and that mixing L1 with (MS-)SSIM [10] gives perceptually better results. We therefore replaced an initial MSE objective with $\alpha\,\mathrm{L1} + (1-\alpha)(1-\mathrm{SSIM})$ and let Optuna select α. Transposed convolutions cause checkerboard artifacts [6], so every decoder uses nearest-neighbour resize followed by a convolution. We also compared a flattened vector bottleneck with a spatial one: the vector version lost layout and produced blurred outputs, so the final models keep an 8×8 spatial latent.

Blind all-in-one restoration. AirNet [3] infers the degradation with a contrastive encoder (MoCo [17]) and conditions the restorer on it. We adapted this idea with a lighter SimCLR NT-Xent objective [7] (no momentum encoder or queue, which is unnecessary at our data scale). Conditioning uses FiLM [8] rather than degradation-guided convolutions, because FiLM

only rescales features inside the bottlenecked path and creates no skip around it.

Mixture of experts. MoE goes back to adaptive local experts [15]. Sparse MoE uses an auxiliary load-balancing loss to prevent collapse [4], [14], and soft MoE replaces discrete routing with continuous weights [16]. We compared a per-sample entropy penalty with a batch-level balance term and chose the latter (Sec. VII). For inpainting-type corruptions, pixel losses produce flat fills. Context Encoders [9] and perceptual losses [12] motivate the VGG feature loss used for the occlusion specialist.

Conditional image translation. Conditional GANs [18], [19] with a U-Net generator [13], a PatchGAN discriminator and an L1 term (pix2pix [5]) are the standard choice for paired translation. We considered injecting the style once (input concatenation) versus at every decoder stage. Measurements showed weak style separation with input-only injection, so the final generator re-injects the embedding through FiLM at every decoder block. Instance normalisation [20] is used in the GAN, and group normalisation [21] in the restoration networks.

Tooling. Hyperparameters are searched with Optuna [1] using TPE [22] and median/collapse pruning. Experiments are tracked with MLflow [23], and models are served with ONNX Runtime [24].

III. Datasets and Corruption Pipeline

A. Oxford-IIIT Pet (Tasks 1–3)

The official trainval collection of Oxford-IIIT Pet [25] (3,680 images) is shuffled with seed 42 and split 80/20 into 2,944 training and 736 validation images. The official test set (3,669 images) is used only for final evaluation. The three sets are disjoint, and train∪val equals the official trainval list exactly. The split is stored once (`data_split.json`) and reused by Tasks 2 and 3. All images are converted to RGB, resized to 128×128 and cached in RAM (361 MB), since reading from the Google Drive FUSE mount made data loading the bottleneck. Fig. 1(a) shows samples.

B. Runtime corruptions

Corruptions are applied inside `Dataset.__getitem__`. Each time an image is loaded, one of four conditions is drawn with equal probability and a fresh severity is sampled (Table I). The label is returned with the image, and no corrupted copies are stored. Over 4,000 loads the empirical frequencies were 25.6% blur, 25.6% salt-and-pepper, 25.1% occlusion and 23.7% clean. All sampled parameters fell within the required ranges. Fig. 1(b) shows the same training index loaded eight times.

C. Deterministic manifests

The validation manifest (one corruption per image, seed $1000+i$) and the test manifest are generated once and stored as JSON. The test manifest gives each test image ten variants: clean plus three severities of each corruption, for 36,690 entries. Every entry stores the corruption type, severity, salt-and-pepper probability, blur kernel and σ, occlusion rectangle coordinates and a seed. Regenerating the occlusion masks from the stored seeds reproduces them exactly. Fig. 1(c) shows every corruption at every severity.

TABLE I
Corruption definitions (training) and fixed test severities

Condition	Training (sampled per load)	Test: low / medium / high
Clean	image unchanged	–
Salt-and-pepper	$p \sim U(0.02, 0.15)$; black/white 50/50	$p = 0.03/0.08/0.15$
Gaussian blur	$k \in \{3,5,7\}$, $\sigma \sim U(0.5, 2.5)$	$(3, 0.7)/(5, 1.5)/(7, 2.5)$
Occlusion	1–3 black rectangles, joint area $U(10\%, 35\%)$, random positions	1/2/3 rectangles, ≈10/20/35%

D. FS2K (Task 4)

FS2K [26] provides 2,104 photo–sketch pairs in three sketch styles with official train (1,058) and test (1,046) definitions. We hold out 15% of the official training portion, stratified by style with seed 42 (`train_test_split`). This gives 899 training pairs (303/298/298 per style) and 159 validation pairs (54/52/53). The test set (619/381/46) is used only for final evaluation. The splits are disjoint. Pairs are resized to 128×128 and scaled to $[-1, 1]$. Training augmentation draws one horizontal flip, one rotation angle in $[-8°, 8°]$ and one resized-crop window per sample, and applies them identically to the photo and its sketch (Fig. 21).

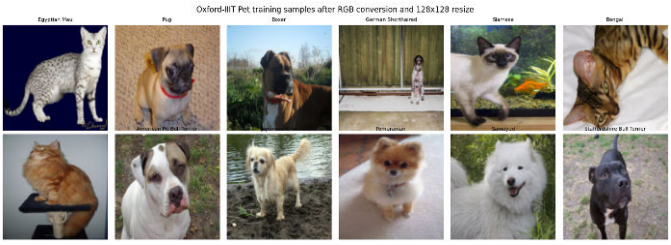
IV. Experimental Setup

Hardware and software. All models were trained on Google Colab with an NVIDIA T4 GPU, using Python 3, PyTorch, `pytorch-msssim`, Optuna (TPE sampler, seed 42), MLflow and ONNX/ONNX Runtime. Checkpoints, manifests, Optuna studies and the MLflow store are kept on Google Drive, and every training loop resumes from its last checkpoint. Global seeds (Python, NumPy, PyTorch) are fixed to 42.

Protocol. Hyperparameters are chosen only on validation data. Optuna trials use short budgets, after which the selected configuration is retrained with early stopping on the validation objective. The best-validation checkpoint is reloaded and evaluated once on the untouched official test set: the 36,690-entry test manifest for Tasks 1–3, and the 1,046 official FS2K test pairs for Task 4.

Metrics.

- Restoration: per-image L1, SSIM (Gaussian window, data range 1) and PSNR $= 10\log_{10}(1/\mathrm{MSE})$, averaged by corruption type and severity. PSNR is capped at 80 dB when an output equals its target exactly.
- Classifier: accuracy, macro and per-class precision/recall/F1, and a row-normalised confusion matrix.
- Soft MoE: mean routing weight per type and severity.



(a) Training samples after RGB conversion and resizing.



(b) `train_ds[0]` loaded eight times: a new corruption and severity on every load.



(c) Fixed test corruptions: every corruption × severity, taken from the deterministic manifest.

Fig. 1. Dataset and corruption pipeline for Tasks 1–3.

- Task 4: L1 (in $[-1, 1]$ units), SSIM and PSNR on $[0, 1]$ images, per style. We also measure style controllability and output diversity (mode-collapse ratio).
- ONNX: maximum and mean absolute difference from PyTorch on real test images.

V. Task 1: Universal Multi-Corruption Denoising Autoencoder

A. Architecture

The model $\hat{x} = D(E(\tilde{x}, c), c)$ (Fig. 2) consists of three parts.

- *Encoder.* A stem convolution, then four stride-2 4×4 convolutions ($128\to64\to32\to16\to8$) with channels $c, 2c, 4c, 8c, 8c$, c=64, GroupNorm and ReLU. One FiLM residual block and dropout follow.
- *Bottleneck.* A 1×1 squeeze convolution produces the latent $z \in \mathbb{R}^{128\times8\times8}$: 8,192 values, 16.7% of the 49,152-value input.
- *Decoder.* A 1×1 expansion, a FiLM residual block and four resize-convolution up-blocks, ending in a sigmoid RGB output.

A degradation encoder (three stride-2 convolutions and global pooling) produces a 64-d code c from the corrupted image itself. This code sets the FiLM (γ, β) parameters, which are initialised to identity. There are *no* encoder–decoder skip connections: the decoder receives only z and the global code c, which has no spatial layout and so cannot carry pixels around the bottleneck. Skips were deliberately not used, because they would copy noise and occlusion masks straight to the output. The model has 20.68 M parameters: 0.11 M in the degradation encoder, 11.81 M in the encoder and 8.77 M in the decoder.

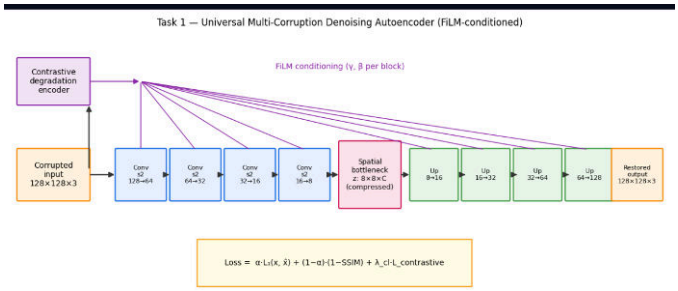


Fig. 2. Task 1 architecture: contrastive degradation encoder, FiLM-conditioned convolutional encoder and decoder, and spatial bottleneck z ($128\times8\times8$).

B. Loss

$$\mathcal{L}_{rec} = \alpha \, \|x - \hat{x}\|_1 + (1 - \alpha)\big(1 - \mathrm{SSIM}(x, \hat{x})\big), \quad (1)$$

$$\mathcal{L}_{total} = \mathcal{L}_{rec} + \lambda_{cl} \, \mathcal{L}_{\text{NT-Xent}}(p_1, p_2), \quad (2)$$

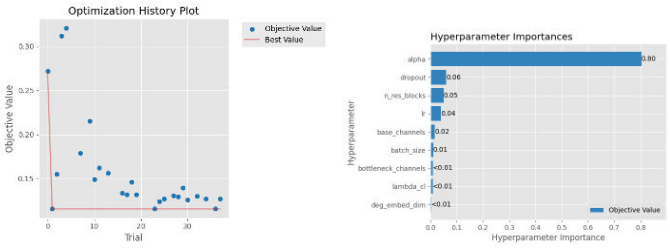
where p_1, p_2 are projections of two random 64×64 crops of the same corrupted image (positives). Crops of other images in the batch serve as negatives (temperature 0.2). Validation and model selection use $\mathcal{L}_{rec}$ only.

C. Hyperparameter optimisation

The Optuna search space covers learning rate, batch size, bottleneck channels, encoder base channels, residual blocks, dropout, α, degradation-embedding dimension and λ_{cl} (Table II). Each trial trains for 4 epochs and is scored by its best validation $\mathcal{L}_{rec}$, which combines reconstruction error

TABLE II
TASK 1 OPTUNA SEARCH SPACE AND SELECTED CONFIGURATION

Hyperparameter	Search range	Selected
Learning rate	$[10^{-4}, 3\times10^{-3}]$ (log)	2.69×10^{-4}
Batch size	{32, 64, 128}	32
Bottleneck channels	{64, 96, 128, 192}	128
Encoder base channels	{24, 32, 48, 64}	64
Residual blocks	[1, 3]	1
Dropout	[0, 0.3]	0.0195
α (L1 weight)	[0.5, 0.95]	0.927
Degradation embed. dim.	{64, 128, 256}	64
λ_{cl}	$[0.02, 0.5]$ (log)	0.0274



(a) Optimisation history. (b) Hyperparameter importance.

Fig. 3. Task 1 Optuna study (38 trials: 25 complete, 12 pruned, 1 failed).

and SSIM. Sampling is TPE with seed 42, a median pruner removes weak trials, and the study is persisted to SQLite. Of 38 trials, 25 completed, 12 were pruned and 1 failed. The best value was 0.1157 (trial 1). Fig. 3 shows the optimisation history and fANOVA importances. α dominates (importance 0.80), followed by dropout (0.06), residual blocks (0.05) and learning rate (0.04). Bottleneck width has importance below 0.01 in the range tested.

D. Training procedure

The selected configuration was trained with Adam (learning rate 2.69×10^{-4}, batch 32) and ReduceLROnPlateau (factor 0.5, patience 4) for up to 60 epochs, with early-stopping patience 10. The best checkpoint was chosen by validation $\mathcal{L}_{rec}$. Fig. 4 shows the curves. Train and validation reconstruction losses fall together from ≈0.20/0.16 to ≈0.05. Validation SSIM rises from 0.40 to 0.78, and the contrastive loss falls from 2.85 to 1.25. The train and validation curves never diverge, so there is no overfitting. Validation SSIM is still rising slowly at epoch 60, which indicates the model is budget-limited rather than capacity-limited.

E. Results

Table III reports L1, SSIM and PSNR on the 36,690-entry test manifest by corruption and severity.

Important caveat. The checkpoint available for this evaluation does not match the training run in Fig. 4. Its test SSIM (0.40–0.41 on all non-occluded conditions) equals the *epoch-1* validation SSIM of the run. Its outputs are nearly identical across clean, noisy and blurred inputs (Fig. 5), and its metadata label ("epoch 10, val. loss 0.0869") is inconsistent with this measured performance. The best checkpoint was

TABLE III
TASK 1 TEST RESULTS BY CORRUPTION TYPE AND SEVERITY (EVALUATED CHECKPOINT)

Type	Severity	L1↓	SSIM↑	PSNR (dB)↑
Clean	–	0.1151	0.4060	16.80
Salt-and-pepper	low	0.1149	0.4060	16.82
	medium	0.1147	0.4058	16.84
	high	0.1149	0.4050	16.84
Gaussian blur	low	0.1152	0.4058	16.79
	medium	0.1153	0.4055	16.79
	high	0.1154	0.4050	16.79
Occlusion	low	0.1223	0.3918	16.19
	medium	0.1318	0.3772	15.47
	high	0.1472	0.3565	14.44
All conditions pooled		0.1207	0.3965	16.38

overwritten during a checkpoint-synchronisation step between Colab sessions. Without retraining, the numbers in Table III therefore describe an early-epoch model; the logged run reached validation SSIM 0.78. Even this early model shows the expected pattern: occlusion is clearly the hardest corruption, and its error grows monotonically with severity (SSIM $0.392\rightarrow0.377\rightarrow0.357$).

F. Failure analysis and discussion

Fig. 5 shows twelve examples with error maps, and Fig. 6 the worst case of each condition.

- *(i) Highly textured images.* The three worst non-occlusion cases are the same image: a dog on gravel in front of a brick wall, with a black frame. Its fine, high-contrast texture cannot pass through an 8×8 latent, and the error map lights up on every texture edge and along the frame.
- *(ii) Large occlusions on dark backgrounds.* The worst occlusion case (SSIM 0.074) hides most of a cat on a black background. The reconstruction is a low-contrast mean colour, and the error concentrates on the masked animal.
- *(iii) Clean inputs.* Clean images lose quality because a universal autoencoder has no identity path. This is the main motivation for the identity bypass in Tasks 2 and 3.

Design lessons: the spatial bottleneck and resize-convolution decoder removed the blur and checkerboard artifacts of earlier versions. Optuna's strong preference for a large L1 weight (α=0.93) favours colour fidelity over structural sharpness. A single universal model must denoise, deblur and inpaint with one set of weights, which limits it on every individual task.

VI. TASK 2: CORRUPTION CLASSIFICATION AND HARD-ROUTED SPECIALISTS

A. Corruption classifier

$C(\tilde{x})$ is a five-stage CNN: stride-2 3×3 convolutions with channels $16\cdot2^i$, GroupNorm and ReLU, followed by global average pooling, dropout and a linear layer. It outputs four logits, giving $p = \mathrm{softmax}(C(\tilde{x}))$ and the route $r = \arg\max_k p_k$. Training minimises multiclass cross-entropy with labels produced by the corruption pipeline. Batches are *exactly* balanced:

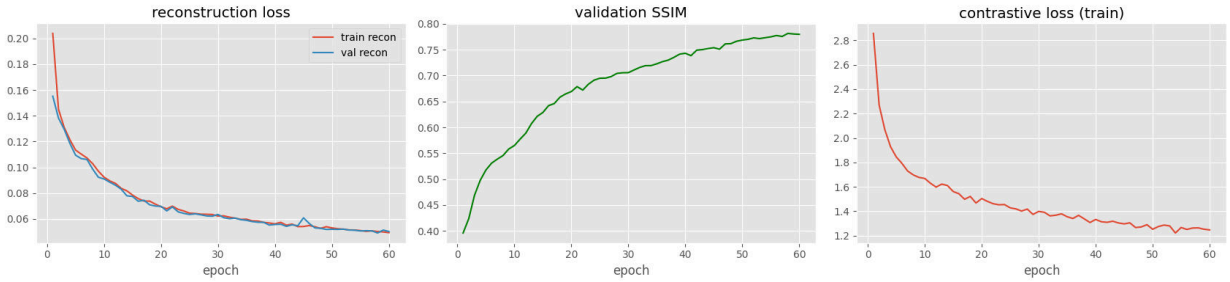


Fig. 4. Task 1 training curves: reconstruction loss (train/validation), validation SSIM and contrastive loss over the 60-epoch run.

TABLE IV
CORRUPTION CLASSIFIER ON THE TEST MANIFEST (36,690 INPUTS)

Class	Precision	Recall	F1	Support
Clean	0.862	0.988	0.921	3,669
Salt-and-pepper	1.000	1.000	1.000	11,007
Gaussian blur	0.996	0.963	0.979	11,007
Occlusion	0.999	0.983	0.991	11,007
Macro average	0.964	0.984	0.973	36,690
Overall accuracy: 0.9827				

the class of training index i is fixed to $i \bmod 4$ while the image and severity are re-sampled on every access, and with an unshuffled loader each batch of 32 holds 8 samples per class (verified on the first five batches). The Optuna search covered learning rate, batch size, depth and base width (the channel configuration), dropout and weight decay. It ran 20 trials (13 complete, 7 pruned, 3 epochs each, objective validation cross-entropy; Fig. 7a) and selected learning rate 6.38×10^{-4}, batch 32, 5 layers, width 16, dropout 0.381 and weight decay 1.02×10^{-4}. The final classifier (0.39 M parameters) was trained with AdamW for 30 epochs (best validation loss 0.0461). Training and validation curves track each other at ≈98% accuracy (Fig. 8a).

Table IV and Fig. 8b report the test performance. Salt-and-pepper is never confused, because isolated pure black and white pixels are unique to it. Almost every error is a corrupted image predicted as *clean*: 404 blurred images, all at low severity (k=3, σ=0.7, a very slight softening at 128×128), and 176 occlusions whose rectangles fall on naturally dark regions such as black fur or shadow. This is why clean has recall 0.988 but precision 0.862. Corrupted classes are almost never confused with each other (only 11 blur↔occlusion errors).

B. Specialist autoencoders

The three specialists share one architecture: the Task 1 backbone without degradation conditioning, with a spatial 128×8×8 bottleneck and no skips. Each has 5.57 M parameters but independently trained weights and a different initialisation seed. Each sees *only* its own corruption during training, with the clean image as the target. One shared Optuna search, using the blur specialist as a proxy because deblurring needs the most high-frequency reconstruction, tuned learning rate, batch size, bottleneck size, base channels, residual blocks, dropout and the L1/SSIM weight α. It ran 20 trials (15 complete, 5 pruned; Fig. 7b) and selected learning rate 1.03×10^{-4}, batch 64, bottleneck 128, base 24, 3 residual blocks, dropout 0.17 and α=0.947. Dropout was searched in $[0, 0.3]$, and the code range was tightened to $[0, 0.15]$ only afterwards. In addition to Eq. (1), the blur specialist adds a Sobel edge loss (weight 0.15), which penalises residual softness. The occlusion specialist adds a VGG-16 perceptual loss [12] (weight 0.10), which discourages flat fills [9]. Each specialist was trained with Adam and ReduceLROnPlateau for up to 200 epochs (patience 25), validated on a dedicated full-size validation manifest for its corruption. The best validation losses were 0.057 (salt-and-pepper), 0.099 (blur) and 0.323 (occlusion, which includes the perceptual term). Fig. 9 shows the curves. Salt-and-pepper and blur converge by ≈150 epochs (validation SSIM ≈0.71–0.72). Occlusion is still improving at epoch 200 (validation SSIM ≈0.65).

C. Hard-routed inference

$$\hat{x} = \begin{cases} \tilde{x} & r = \text{clean (identity bypass)} \\ A_{salt}(\tilde{x}) & r = \text{salt} \\ A_{blur}(\tilde{x}) & r = \text{blur} \\ A_{occ}(\tilde{x}) & r = \text{occlusion} \end{cases} \tag{3}$$

Clean predictions skip all experts (Fig. 10). In *oracle* mode the manifest label selects the expert; in *predicted* mode the classifier does. Table V lists both by corruption and severity. The oracle columns are each specialist's own performance on its corruption at each severity level. Oracle and predicted routing are identical on every salt-and-pepper, medium/high-blur and most occlusion inputs. Predicted routing is even *better* on low blur (SSIM 0.755 vs 0.721) and low occlusion (0.696 vs 0.688). In these cases the misrouted inputs are barely corrupted, so the identity bypass keeps them closer to the target than the specialist reconstruction does. On clean inputs predicted routing loses 0.0023 SSIM, because 45 clean images (1.2%) were sent to an expert. Pooled over all 36,690 inputs, SSIM is 0.718 (oracle) and 0.723 (predicted).

D. Classifier-induced failures

636 test inputs (1.73%) are misrouted (Fig. 11). Their mean SSIM (0.908) is *higher* than that of correctly routed inputs

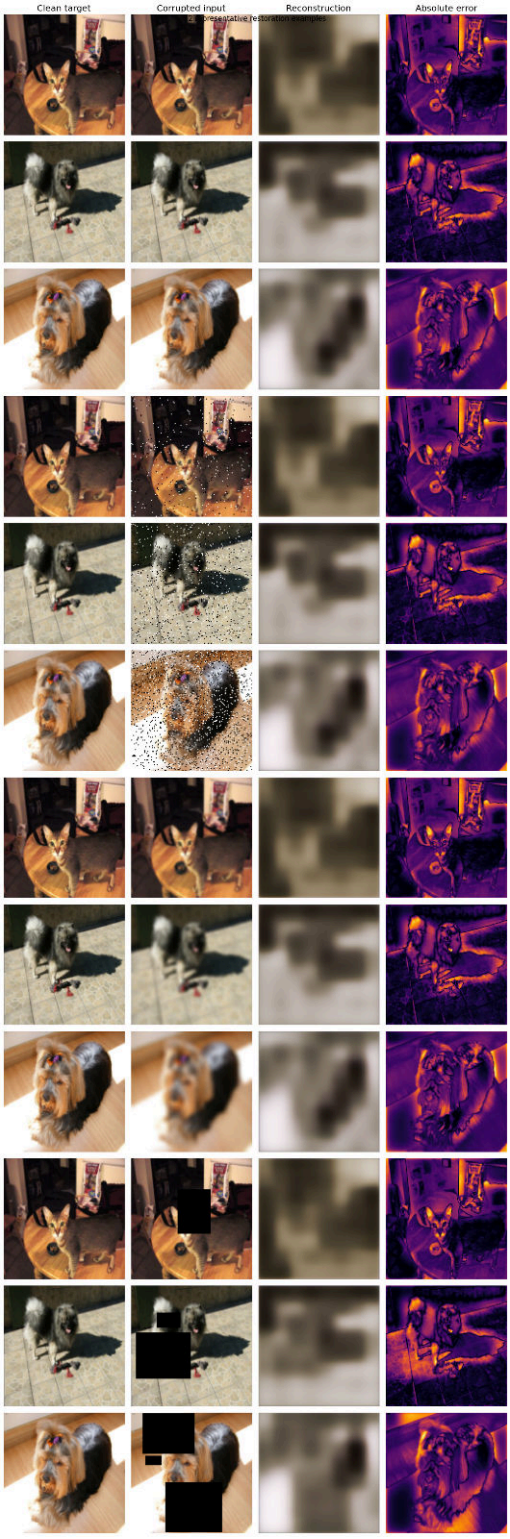


Fig. 5. Twelve representative Task 1 test examples (3 clean, 3 salt-and-pepper, 3 blur, 3 occlusion). Columns: clean target, corrupted input, reconstruction, absolute error map.

(0.719), because only the mildest corruptions get misrouted. Two failure modes matter.

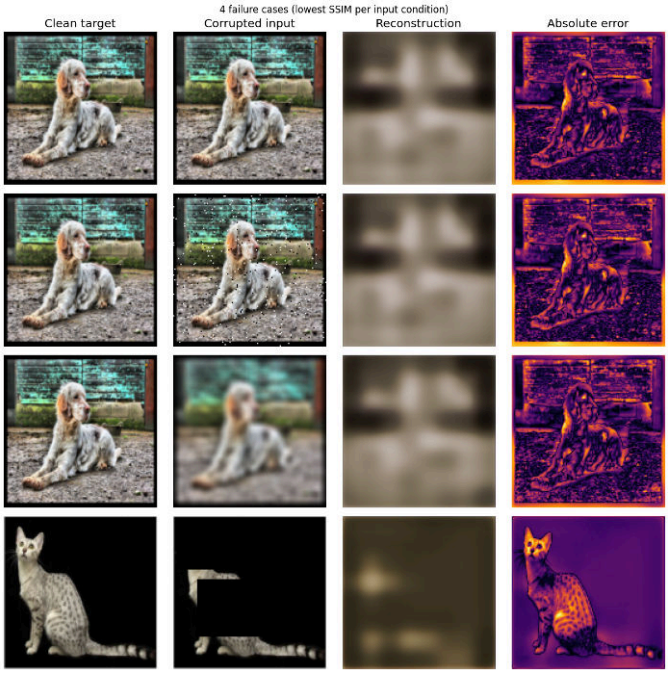


Fig. 6. Task 1 failure cases: the lowest-SSIM sample of each input condition (clean SSIM 0.103, salt-and-pepper 0.103, blur 0.099, occlusion 0.074), with absolute error maps.

TABLE V
HARD ROUTING ON THE TEST SET: ORACLE VS PREDICTED ROUTING, AND COMPARISON WITH TASK 1 (SSIM; PSNR IN DB)

		SSIM↑			PSNR↑	
Type	Sev.	Task 1	Oracle	Pred.	Oracle	Pred.
Clean	–	0.406	1.000	0.998	80.0*	79.4*
S&P	low	0.406	0.706	0.706	24.24	24.24
	med.	0.406	0.703	0.703	24.20	24.20
	high	0.405	0.695	0.695	24.05	24.05
Blur	low	0.406	0.721	0.755	24.38	25.34
	med.	0.406	0.716	0.716	24.43	24.43
	high	0.405	0.695	0.695	24.09	24.09
Occl.	low	0.392	0.688	0.696	22.72	22.62
	med.	0.377	0.654	0.655	21.47	21.47
	high	0.357	0.607	0.607	19.94	19.94

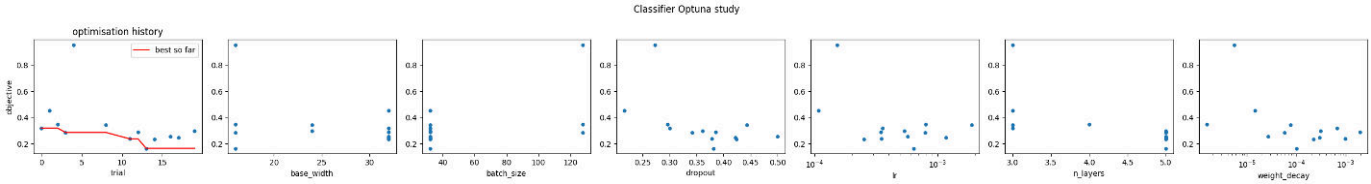
*PSNR of an unchanged image is capped at 80 dB (MSE= 0).

- *Occlusion predicted as clean* (176 cases). The black rectangle survives unchanged (row 2).
- *Clean image predicted as blur* (37 cases). The image is needlessly passed through the blur specialist and loses detail (row 3).

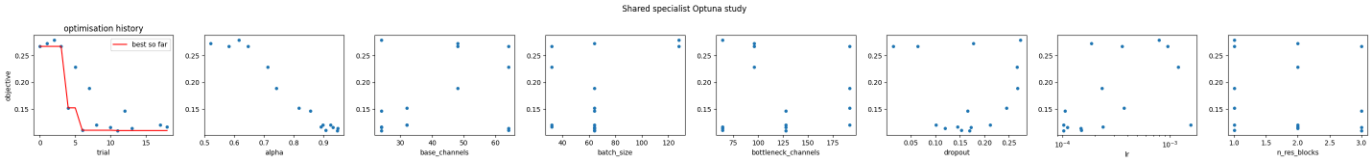
The cheapest error is a low-severity blur sent to the bypass (rows 1 and 4), where the output keeps a slight softness but avoids the specialist's own smoothing. Fig. 13 shows the complete operational pipeline on test images, as displayed by the application.

E. Comparison with Task 1

Fig. 12 and Table V show that the specialists beat the evaluated Task 1 checkpoint on every condition, by about

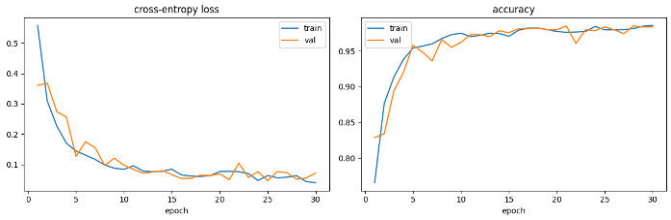


(a) Classifier study (20 trials).

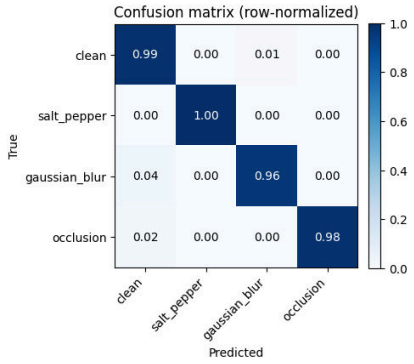


(b) Shared specialist study (20 trials, blur specialist as proxy).

Fig. 7. Task 2 Optuna studies: optimisation history (red: best so far) and objective versus each hyperparameter.



(a) Training curves.



(b) Row-normalised confusion matrix (test).

Fig. 8. Task 2 corruption classifier.

+0.29 SSIM and +7.4 dB PSNR on salt-and-pepper and blur. Specialists help because each learns one inverse mapping and clean images bypass restoration entirely. They fail when the classifier is wrong and when an input mixes corruptions, because only one expert is ever applied. Part of the gap is also due to the early-epoch Task 1 checkpoint (Sec. V): the Task 1 training run reached validation SSIM 0.78. The ONNX exports reproduce PyTorch with maximum absolute differences of 8.1×10^{-6} (classifier logits, 100% argmax agreement) and 2.6×10^{-5}, 2.1×10^{-5} and 2.1×10^{-5} (salt-and-pepper, blur and occlusion specialists) on 32 test images.

VII. Task 3: Jointly Trained Soft Mixture of Experts

A. Architecture

The gate G, initialised from the Task 2 classifier, produces temperature-scaled weights over four branches (Fig. 14):

$$w = \mathrm{softmax}\big(G(\tilde{x})/\tau\big), \quad w = [w_0, w_1, w_2, w_3], \tag{4}$$

$$\hat{x} = w_0\tilde{x} + w_1 A_{salt}(\tilde{x}) + w_2 A_{blur}(\tilde{x}) + w_3 A_{occ}(\tilde{x}). \tag{5}$$

The experts are initialised from the three Task 2 specialists, and w_0 weights an identity branch. The weighted sum is differentiable, so the reconstruction loss trains both the gate and the experts. The system has 17.1 M parameters (gate 0.39 M and 3×5.57 M).

B. Joint loss and training

$$\mathcal{L}_{MoE} = \lambda_1 \mathrm{L1} + \lambda_2(1-\mathrm{SSIM}) + \lambda_c\,\mathrm{CE}\big(G(\tilde{x}), y\big) + \lambda_b\,\mathcal{L}_{bal}, \tag{6}$$

$$\mathcal{L}_{bal} = \sum_{k=0}^{3}\Big(\bar{w}_k - \tfrac{1}{4}\Big)^2, \tag{7}$$

with $\lambda_1{=}\alpha$, $\lambda_2{=}1{-}\alpha$ and $\bar{w}_k$ the batch-mean weight of branch k. Batches are exactly class-balanced ($i \bmod 4$), so a uniform batch-average target of $\frac{1}{4}$ is correct even though individual images should be routed sharply. This is the load-balancing principle of sparse MoE [4], [14]. We rejected a per-sample entropy penalty, because it would push unambiguous inputs towards a uniform mixture. The cross-entropy term keeps G tied to the runtime corruption label, which is used only during training.

Warm-up. For 10 epochs only the gate is trained (Adam, learning rate 10^{-3}, $\tau{=}1$, $\alpha{=}0.8$, $\lambda_c{=}0.1$, $\lambda_b{=}0.01$), and the experts stay frozen at their Task 2 weights. After warm-up the mean validation weights were 0.256/0.272/0.253/0.219, so no branch collapsed.

Joint fine-tuning. All parameters are unfrozen and trained with the Optuna-selected learning rate of 1.61×10^{-5}, 62× smaller than in warm-up, for 60 epochs (patience 15). Fig. 15b

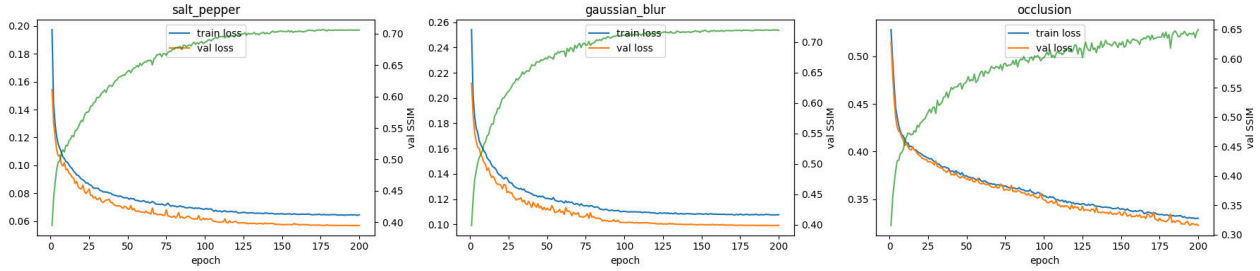


Fig. 9. Training/validation loss (left axis) and validation SSIM (right axis, green) of the three specialists.

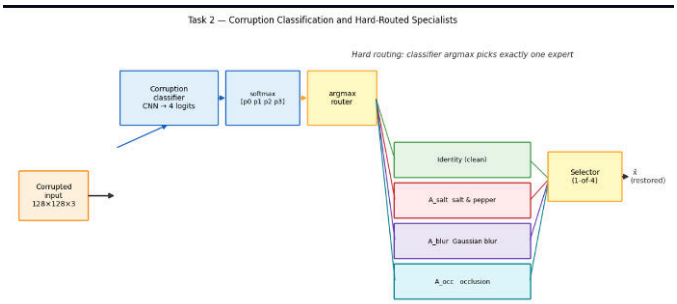


Fig. 10. Task 2 architecture: the classifier's argmax selects exactly one branch (identity bypass or one specialist).

shows validation loss falling from 0.052 to 0.049 and validation SSIM rising from 0.783 to 0.802, while gate accuracy stays at 97.5–98.4%.

Optuna. The search covered the joint learning rate, τ, α, λ_c and λ_b (Fig. 15a). It ran 15 trials of 3 epochs, each starting from the post-warm-up checkpoint. Trials were pruned by the median pruner *and* on routing collapse (any branch's mean weight < 0.02 or > 0.9): 7 completed and 8 were pruned. Selected: learning rate 1.61×10^{-5}, τ=1.692, α=0.923, λ_c=0.0399, λ_b=0.00126.

C. Reconstruction results

Table VI compares all restoration systems on the identical test manifest. The soft MoE beats hard routing (oracle and predicted) on *every* corrupted type and severity. The gains are largest where the corruption is mild or ambiguous: low occlusion $+0.106$ SSIM over predicted routing, low blur $+0.072$ and medium occlusion $+0.072$. On high salt-and-pepper, where one expert is already near-certain, the gain is small ($+0.002$). Clean inputs stay almost untouched (SSIM 0.997) because the identity branch carries 95% of the weight. Pooled over the test manifest the soft MoE reaches L1 0.0392, SSIM 0.764 and PSNR 27.50 dB. Fig. 16 shows the qualitative difference: the soft output blends in part of the input, which keeps fine detail and colour that the hard-routed expert smooths away.

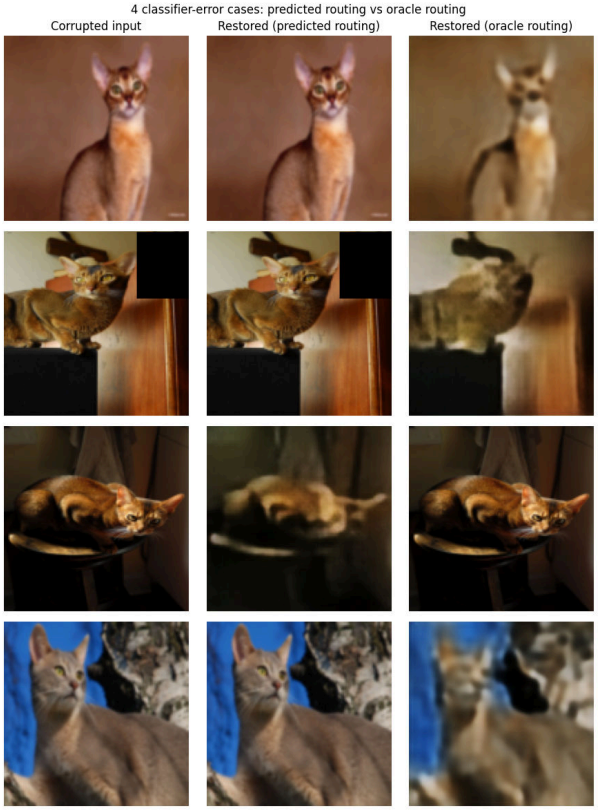


Fig. 11. Classifier errors that change the restoration: corrupted input, output under predicted routing, output under oracle routing.

D. Gating behaviour

Table VII gives the mean weights for every true type and severity, and Fig. 17 the routing heatmap. Every row peaks on its matching branch: clean 0.954, salt-and-pepper 0.931, blur 0.755 and occlusion 0.804. Routing becomes sharper as severity increases (own-expert weight on blur: 0.68, 0.79, 0.79; on occlusion: 0.72, 0.83, 0.87), while mild corruptions keep a substantial identity weight (0.32 on low blur, 0.26 on low occlusion). This smooth, severity-dependent hedging is exactly what hard routing cannot express, and it explains the gains in Table VI. In the low-entropy examples (Fig. 18a) a single branch takes > 0.99 of the weight. In the high-entropy examples (Fig. 18b) mild blur and occlusion inputs

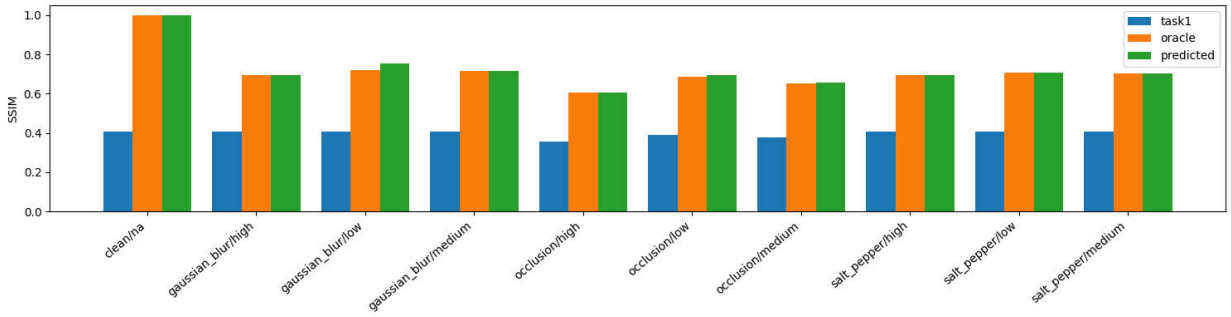


Fig. 12. Test SSIM by corruption/severity: Task 1 universal autoencoder (evaluated checkpoint) vs Task 2 oracle and predicted hard routing.



Fig. 13. End-to-end hard routing reloaded from the saved checkpoints: clean target, corrupted input, classifier verdict (four probabilities, predicted class, selected expert, inference time), restored output.

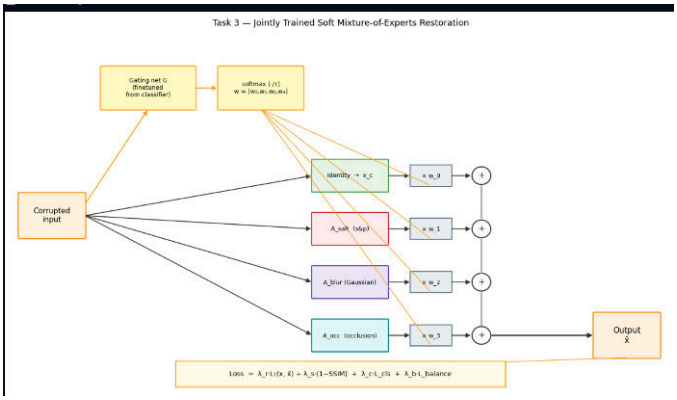


Fig. 14. Task 3 soft mixture of experts: a gate (initialised from the Task 2 classifier) weights the identity branch and three experts (initialised from the Task 2 specialists).

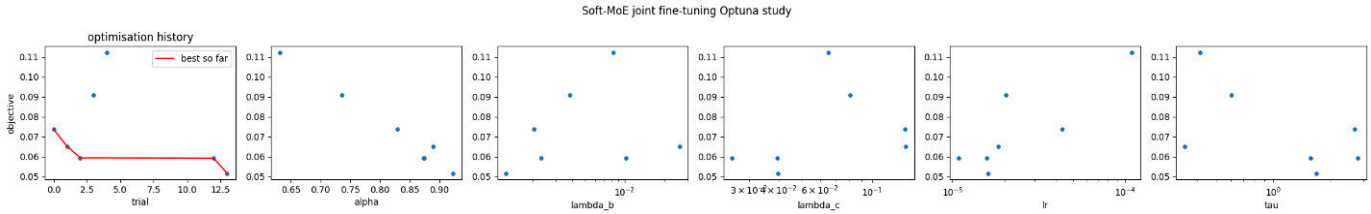
TABLE VI
TEST SSIM / PSNR (DB) OF ALL RESTORATION SYSTEMS

		SSIM↑				PSNR↑	
Type	Sev.	T1	T2-or.	T2-pr.	**T3**	T2-pr.	**T3**
Clean	–	0.406	1.000	0.998	0.997	79.4*	57.3
	low	0.406	0.706	0.706	**0.745**	24.24	**25.16**
S&P	med.	0.406	0.703	0.703	**0.715**	24.20	**24.57**
	high	0.405	0.695	0.695	**0.697**	24.05	**24.31**
	low	0.406	0.721	0.755	**0.827**	25.34	**26.73**
Blur	med.	0.406	0.716	0.716	**0.756**	24.43	**25.40**
	high	0.405	0.695	0.695	**0.716**	24.09	**24.71**
	low	0.392	0.688	0.696	**0.803**	22.62	**24.23**
Occl.	med.	0.377	0.654	0.655	**0.726**	21.47	**22.28**
	high	0.357	0.607	0.607	**0.656**	19.94	**20.36**

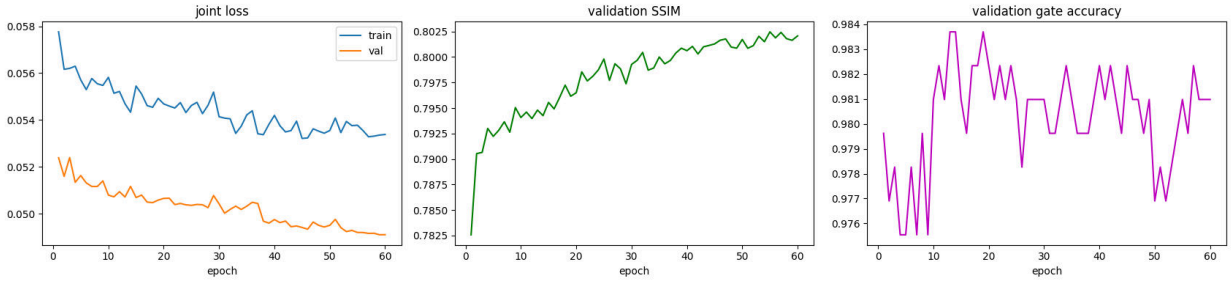
T1: Task 1 (evaluated checkpoint); T2-or./pr.: oracle/predicted hard routing; T3: soft MoE.

split weight between the identity, blur and occlusion branches (e.g. 0.41/0.31/0.28).

Inactive or dominating experts. The dataset-wide mean weights are 0.243 (identity), 0.281 (salt-and-pepper), 0.233 (blur) and 0.243 (occlusion), all close to $\frac{1}{4}$, so no expert is inactive. No expert dominates unrelated inputs: the salt-and-pepper expert receives at most 0.010 on any other type, the blur expert at most 0.035 (on clean images) and the occlusion expert at most 0.009. The only notable cross-talk is the identity branch on mild blur and occlusion, which is



(a) Optuna study (15 trials; 8 pruned, including routing collapse).



(b) Joint fine-tuning curves.

Fig. 15. Task 3 hyperparameter search and joint training.

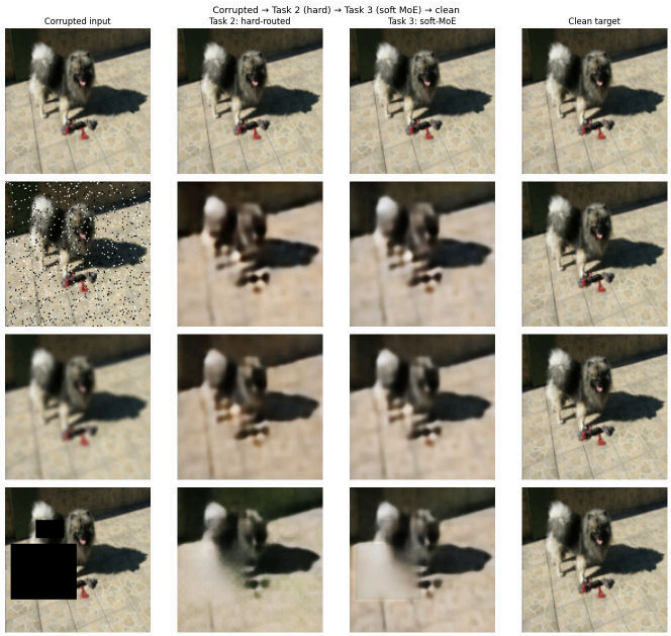


Fig. 16. Corrupted input, Task 2 hard routing, Task 3 soft MoE and clean target (clean, salt-and-pepper, blur and occlusion inputs).

intended behaviour.

E. Failure cases and limitations

Fig. 19 shows the lowest-SSIM output for each corruption: high salt-and-pepper (0.381) and high blur (0.339) on a heavily textured field of flowers, and high occlusion (0.373), where most of a dog is masked. The occlusion output shows the main limitation of pixel-space mixing. With 0.12 identity weight, a faint trace of the black rectangles remains, and the expert's fill is a smooth, low-detail patch. Further limitations:

TABLE VII
MEAN ROUTING WEIGHTS BY TRUE CORRUPTION TYPE AND SEVERITY (TEST)

Type	Sev.	w_0 (id.)	w_1 (S&P)	w_2 (blur)	w_3 (occl.)
Clean	–	**0.954**	0.001	0.035	0.009
S&P	low	0.117	**0.880**	0.000	0.003
	med.	0.045	**0.954**	0.000	0.001
	high	0.040	**0.958**	0.000	0.001
Blur	low	0.317	0.000	**0.681**	0.002
	med.	0.211	0.000	**0.788**	0.001
	high	0.205	0.000	**0.794**	0.001
Occl.	low	0.261	0.010	0.014	**0.716**
	med.	0.160	0.005	0.008	**0.826**
	high	0.120	0.003	0.006	**0.871**

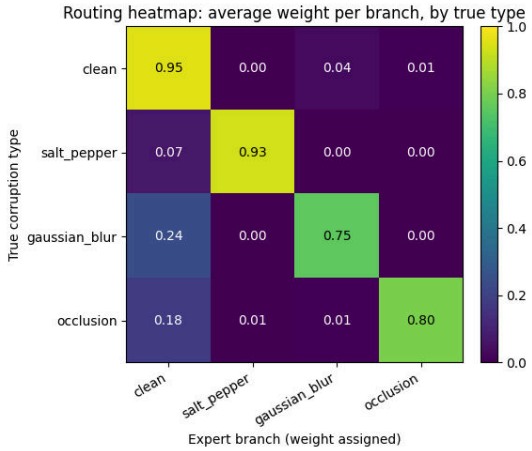
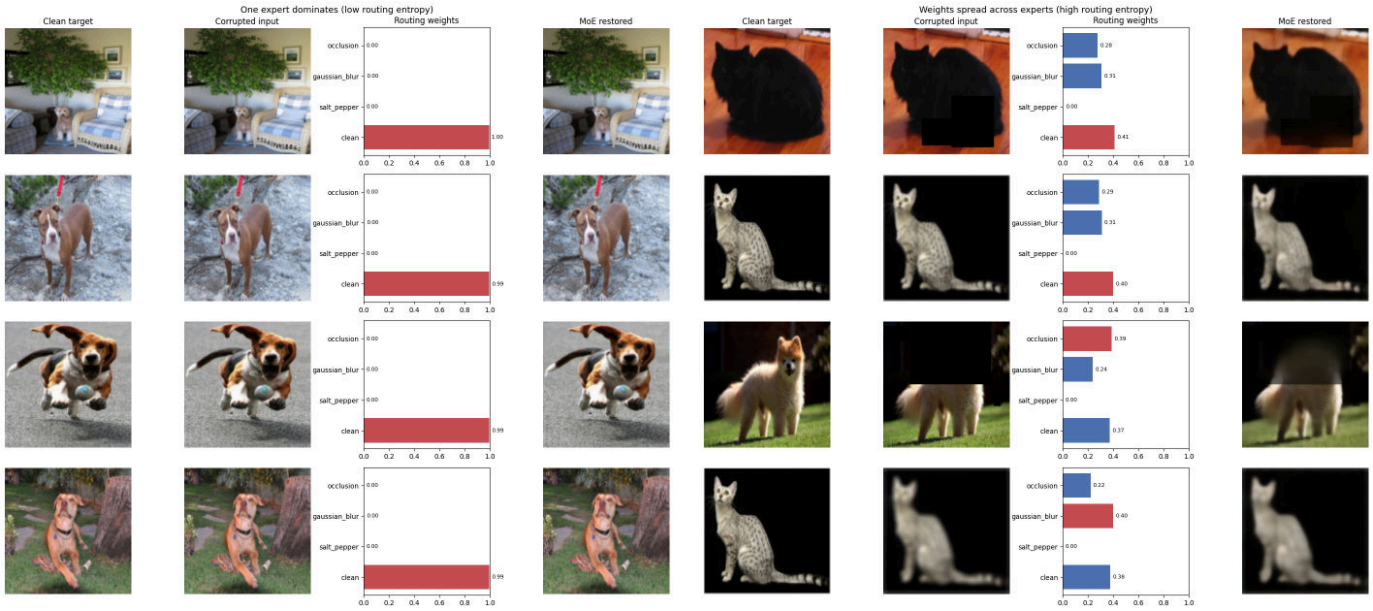


Fig. 17. Routing heatmap: mean branch weight per true corruption type.

- All three experts run on every input, about 3× the compute of hard routing.



(a) One expert dominates (lowest routing entropy).

(b) Weights spread across experts (highest routing entropy).

Fig. 18. Soft-MoE routing examples: clean target, corrupted input, routing weights (red: strongest branch) and output.

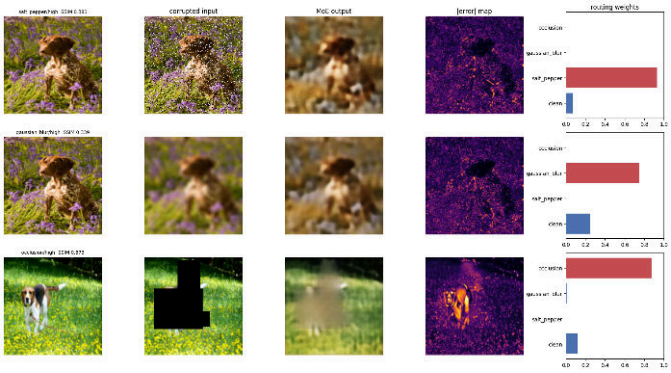


Fig. 19. Task 3 failure cases (lowest SSIM per corruption) with error maps and routing weights.

- The balance target assumes a balanced corruption prior.
- The gate was trained on single corruptions only, so behaviour on genuinely mixed corruptions is extrapolation.

The single-graph ONNX export (gate, three experts and weighted sum) reproduces PyTorch to $4.8{\times}10^{-7}$ (image) and $2.4{\times}10^{-7}$ (weights).

VIII. Task 4: Style-Conditioned Face-to-Sketch cGAN

A. Architecture and losses

The generator $\hat{y} = G(x, s)$ is a five-level U-Net (Fig. 20).

- The style $s \in \{0, 1, 2\}$ is mapped by a learned embedding (dimension 16). The embedding is concatenated with the photo at the input and with the bottleneck features, and it is re-injected by FiLM in *every* decoder block.
- Upsampling uses resize-convolution, and dropout is used in the deepest decoder blocks.
- The output layer is a tanh.

The PatchGAN discriminator $D(x, y, s)$ receives the photo, a real or generated sketch and the broadcast style embedding (its own learned embedding), and outputs a $14{\times}14$ map of patch logits. Losses use BCE with logits:

$$\mathcal{L}_D = \tfrac{1}{2}\big[\mathrm{BCE}(D(x, y, s), 1) + \mathrm{BCE}(D(x, G(x, s), s), 0)\big], \tag{8}$$

$$\mathcal{L}_G = \mathrm{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1}\|y - G(x, s)\|_1. \tag{9}$$

With the selected configuration, G has 7.12 M parameters and D 1.57 M. The style conditioning is effective: the same photo produces clearly different sketches for different styles (Fig. 24).

B. Hyperparameter search and training

The Optuna search (Fig. 22a) tuned the generator and discriminator learning rates, batch size, base channels, dropout, style-embedding dimension and $\lambda_{L1} \in [20, 200]$ around the pix2pix default of 100. It ran 15 shortened trials of 5 epochs, objective validation L1, with pruning on the median rule, mode collapse (output-diversity ratio < 0.15) and an overpowering discriminator (>99% real and fake accuracy). Ten trials completed and five were pruned. Selected: G learning rate $4.54{\times}10^{-4}$, D learning rate $6.83{\times}10^{-5}$, batch 8, base 48, dropout 0.195, embedding 16 and λ_{L1}=137.3.

The selected configuration was then retrained on the full schedule: 10 epochs of L1-only generator pre-training (a pix2pix stabilisation trick), then adversarial training with Adam (β_1=0.5) for up to 150 epochs, with early stopping (patience 25) on validation L1. Training was resumed across

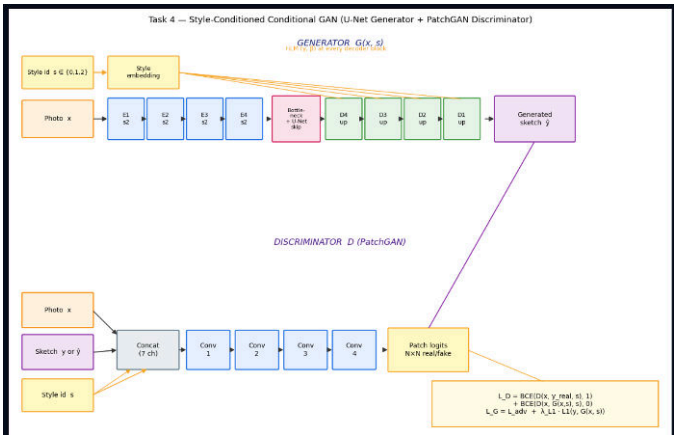


Fig. 20. Task 4: style-conditioned U-Net generator (FiLM at every decoder block) and style-conditioned PatchGAN discriminator.



(a) Training pairs, two per style. (b) Identical paired augmentation.

Fig. 21. FS2K data at 128×128: photo–sketch pairs and identical spatial augmentation of both members of a pair.

Colab sessions and stopped at epoch 59, with the best validation L1 of 0.1895 (in $[-1, 1]$ units). Fig. 22b records the discriminator real and fake losses, the generator adversarial and reconstruction losses and the validation measurements *separately*. The discriminator stays in a healthy band (real/fake accuracy ≈0.76/0.82, losses ≈0.47/0.42) without overpowering the generator. Training L1 keeps decreasing while validation L1 plateaus at ≈0.19, which is the reason for early stopping. Fig. 23 shows the same six validation photos logged at fixed intervals: blurry, low-contrast strokes at epoch 1 become clean, style-specific line work later.

C. Results

Table VIII reports the official test set results, generated with each pair's true style. Style 3 is reproduced best (SSIM 0.620, PSNR 18.9 dB). Style 2's dense, high-contrast cross-hatching is hardest for a pixel metric (SSIM 0.391), since small stroke misalignments are penalised twice. To measure style controllability, each test photo is rendered in all three styles and we record which output is closest (L1) to the ground truth (Fig. 25b). The true style wins for 97% of Style 2 and 98% of Style 3 photos, and for 73% of Style 1 photos; the rest are closest to Style 3, its most similar light pencil style. Weighted over the test set, the true style wins for ≈83% of photos. The mode-collapse diversity ratios on 16 test photos are 0.46, 0.82 and 0.56 (collapse threshold 0.25), so the generator follows its input in all styles. Fig. 24 shows two test photos of every ground-truth style rendered in all three styles. The ONNX generator matches PyTorch to $\leq 4.0\times10^{-6}$ for all styles.

TABLE VIII
TASK 4 RESULTS ON THE OFFICIAL FS2K TEST SET (1,046 PAIRS)

Style (test pairs)	L1† ↓	SSIM↑	PSNR (dB)↑
Style 1 (619)	0.204	0.464	15.93
Style 2 (381)	0.316	0.391	12.14
Style 3 (46)	0.127	0.620	18.87
Overall	0.241	0.444	14.68

†L1 in $[-1, 1]$ units; SSIM and PSNR on $[0, 1]$ images.

TABLE IX
ONNX RUNTIME VS PYTORCH (REAL TEST INPUTS)

Model	Max $\lvert\Delta\rvert$	Mean $\lvert\Delta\rvert$
Task 1 universal AE	4.5×10^{-6}	1.6×10^{-7}
Task 2 classifier (logits)	8.1×10^{-6}	argmax 100%
Task 2 salt-and-pepper / blur / occl.	$2.6/2.1/2.1\times10^{-5}$	$\leq 3.0\times10^{-7}$
Task 3 soft MoE (image / weights)	$4.8/2.4\times10^{-7}$	–
Task 4 generator (3 styles)	$\leq 4.0\times10^{-6}$	$\leq 1.7\times10^{-7}$

D. Failure analysis

Three failure types appear.

- *Artifacts.* Hair is the dominant error region in every style (Fig. 25a), because artists draw strands freely and the L1 term averages them into soft grey texture. Faint checkerboard texture remains in dark clothing.
- *Style confusion.* Style 1 sometimes collapses towards the heavier Style 3 rendering (26%).
- *Pairing errors.* The two worst Style 1 cases pair a man's photo with a woman's sketch. The file resolver falls back to matching sketches by base filename when the per-folder path does not match. FS2K restarts numbering in each of its three sub-folders, so some photos are paired with a sketch from another folder.

These mismatched pairs inflate the measured Style 1 error and add noise to training. Matching sketches by *folder index plus id* fixes the problem, and this is the first item of future work. No mode collapse was observed.

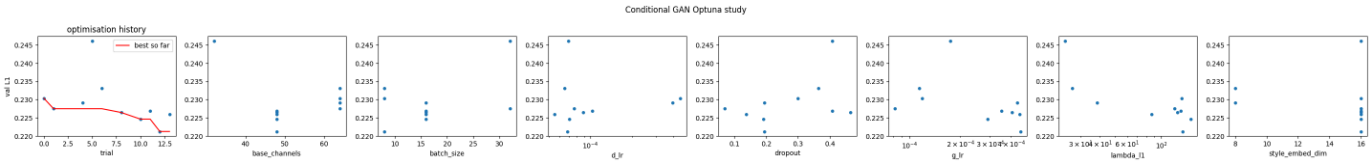
IX. ONNX EXPORT, APPLICATION AND DEPLOYMENT

A. ONNX export and verification

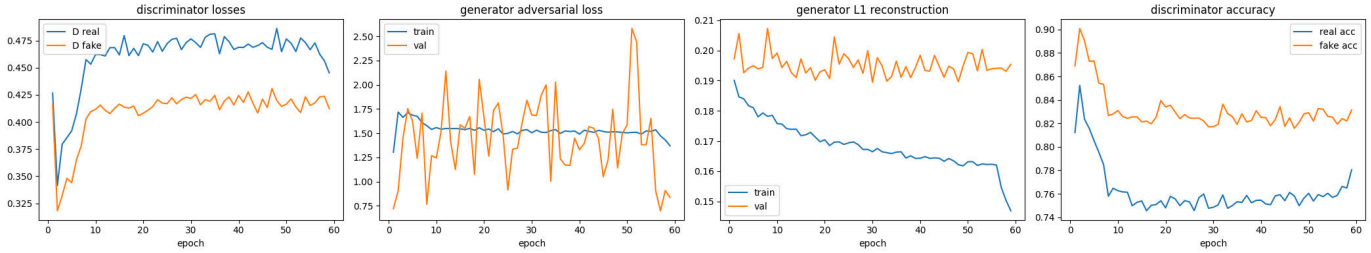
The following models are exported with opset 17 and dynamic batch size:

- the Task 1 universal autoencoder;
- the Task 2 classifier and three specialists;
- the complete Task 3 soft-MoE pipeline as a single graph with two outputs (restored image and routing weights);
- the Task 4 generator, with inputs photo and `style_idx`. The discriminator is training-only and is not exported.

Each export is checked with `onnx.checker` and compared with PyTorch on real test inputs (Table IX).

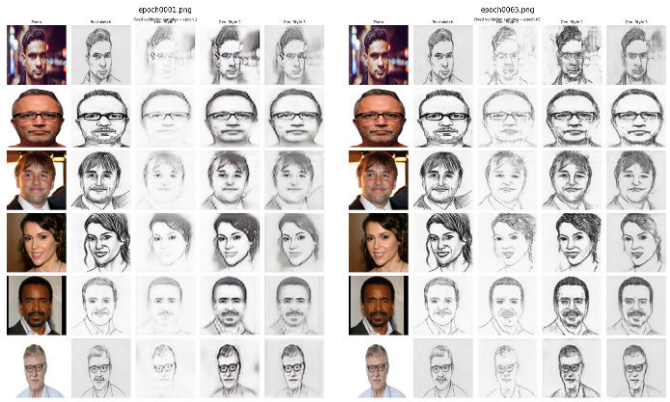


(a) Optuna study (15 trials; 5 pruned).



(b) Separately recorded losses and discriminator accuracies.

Fig. 22. Task 4 hyperparameter search and adversarial training.



(a) Epoch 1. (b) Epoch 65 (last logged grid).

Fig. 23. Fixed validation photos rendered in all three styles during training (photo, real sketch, generated Styles 1–3).

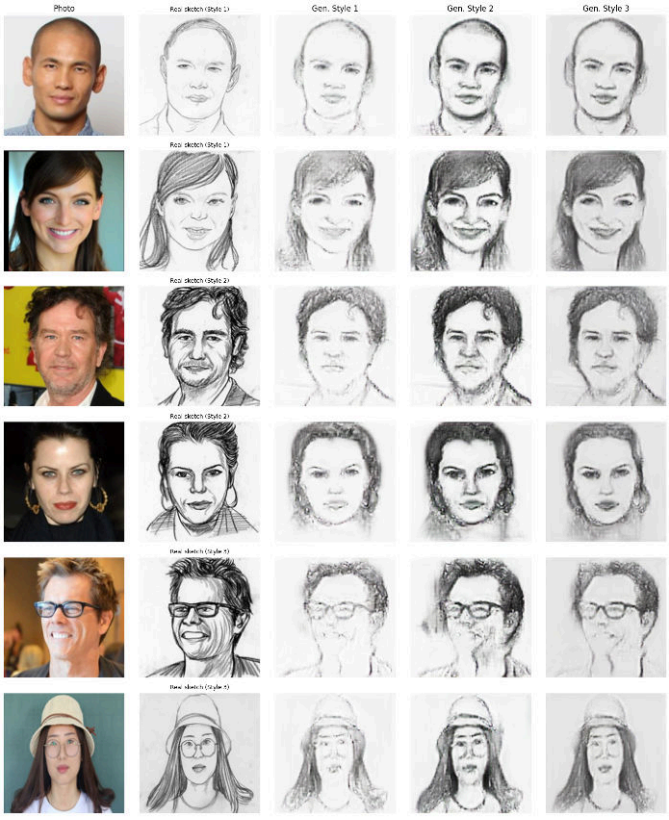


Fig. 24. Official test photos (two per ground-truth style): photo, real sketch and generated Styles 1–3.

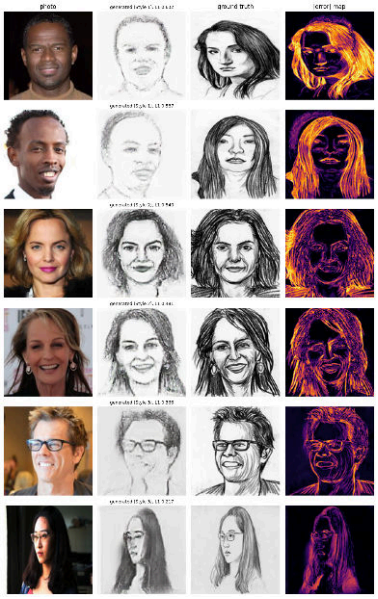
B. Interface design and implementation

The interface was first designed in Google Stitch. Fig. 27 shows the original Stitch canvas: the prompt history and the generated *Universal Restoration* screen with the four workspace tabs, degradation and severity controls, side-by-side input and reconstruction panels, and verdict and timing cards. The telemetry values in this mock-up are Stitch's placeholders, not measurements. The implementation follows its layout, card structure and controls in a single React application styled with Tailwind CSS. It has four tabs, one per workspace (Fig. 28):

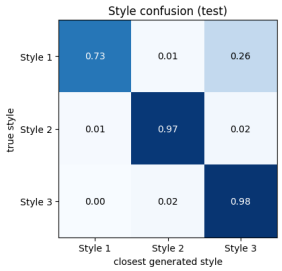
- *Universal Restoration*: upload, corruption type and severity, input/corrupted/restored panels, the selected settings, inference time and download.
- *Hard-Routed Restoration*: the four class probabilities, predicted corruption, selected expert, output and time.
- *Soft Mixture-of-Experts Restoration*: all four routing weights, with the dominant expert highlighted, and the output and time.
- *Face-to-Sketch Generator*: upload or webcam capture, Style 1/2/3 selection, side-by-side display and download.

The FastAPI back end exposes `GET /health` and `POST /universal-restoration`, `/hard-routing`, `/soft-moe` and `/face-to-sketch`. It works as follows:

- Uploads are validated by type (PNG, JPEG, WEBP or BMP), size ($\leq$10 MB) and decodability, and rejected with



(a) Failure cases with error maps.



(b) Style confusion.

Fig. 25. Task 4 failure analysis: (a) the two highest-L1 test outputs per style; (b) which generated style is closest to the ground truth.

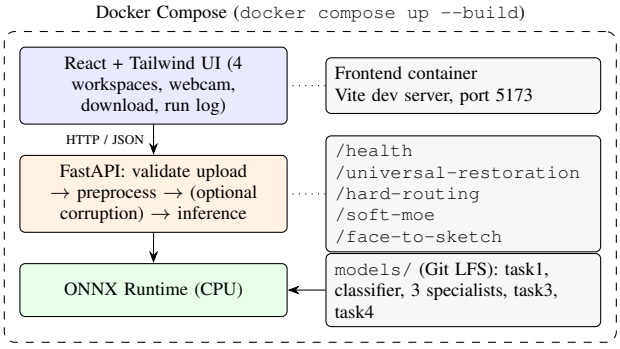


Fig. 26. Application architecture: browser front end, FastAPI back end and ONNX Runtime models, started with one Docker Compose command.

HTTP 400/413.

- Images are preprocessed exactly as in training: RGB, 128×128, $[0, 1]$ for Tasks 1–3 and $[-1, 1]$ for Task 4.
- The required ONNX models are lazy-loaded with ONNX Runtime.
- Corruptions can be applied server-side at the fixed test severities.
- Responses return the images together with probabilities or routing weights, the selected expert and the inference time.

The universal and soft-MoE endpoints always run their full model; the identity branch, not an external classifier, handles clean inputs in Task 3.

C. Containerised deployment

Fig. 26 shows the deployed architecture. The back end (`python:3.11-slim`, ONNX Runtime) and the front end (`node:20`, Vite) each have a Dockerfile. `docker compose up --build` starts both, with the models mounted read-only from `models/` (stored with Git LFS). The application is then available at `http://localhost:5173`, with the API documentation at `http://localhost:8000/docs`. No IDE or manual Python step is needed. CPU inference times observed in the application were 212 ms (Task 1), 194 ms (Task 2), 969 ms (Task 3, four networks) and 54–252 ms (Task 4). The first request includes model loading.

D. Experiment tracking

Each notebook logs to one MLflow tracking store, kept on Google Drive (SQLite plus artifact directory), in four experiments (`Task1-Universal-AE`, `Task2-Hard-Routing`, `Task3-Soft-MoE`, `Task4-Face2Sketch-cGAN`). Each run records:

- the selected hyperparameters and per-epoch training/validation histories;
- every Optuna trial as a nested run;
- test metrics by corruption, severity or style;
- the best checkpoints, ONNX files, figures and fixed validation samples.

The UI is served from Colab and shown in the demonstration video. The Optuna studies are stored as SQLite (Task 1) and as JSON trial logs (Tasks 2–4).

X. Discussion

Universal vs hard vs soft. On identical data and manifests, quality improves at each stage: universal (evaluated checkpoint) → hard routing (+0.29 SSIM on salt-and-pepper and blur) → soft MoE (a further +0.002 to +0.11). The soft system wins because it can *hedge*: mild corruptions keep part of the input through the identity branch instead of being fully re-synthesised by a smoothing autoencoder.

Difficulties and how they were resolved.

- Blurry outputs were traced to three causes, each fixed with evidence: an MSE objective (replaced by L1+SSIM [11]), transposed convolutions (replaced by resize-convolution [6]) and a flattened bottleneck (made spatial).
- Training that appeared to hang on Colab was Drive I/O, solved by caching the dataset in RAM.
- Specialists stopped too early because of a noisy quarter-size validation signal. Dedicated full-size validation manifests per corruption, plus a larger budget, fixed it.
- Overfit sanity checks (memorising eight pairs) and output-diversity checks separated architectural bugs from undertraining.
- In the GAN, input-only style conditioning was too weak (verified by measuring output differences between styles) and was replaced by per-block FiLM.

XI. Conclusion, Limitations and Future Work

Task 1. A blind universal autoencoder with a spatial bottleneck and contrastive FiLM conditioning trains stably (validation SSIM 0.78). The checkpoint available for evaluation is early-epoch (test SSIM 0.40), and retraining with the logged

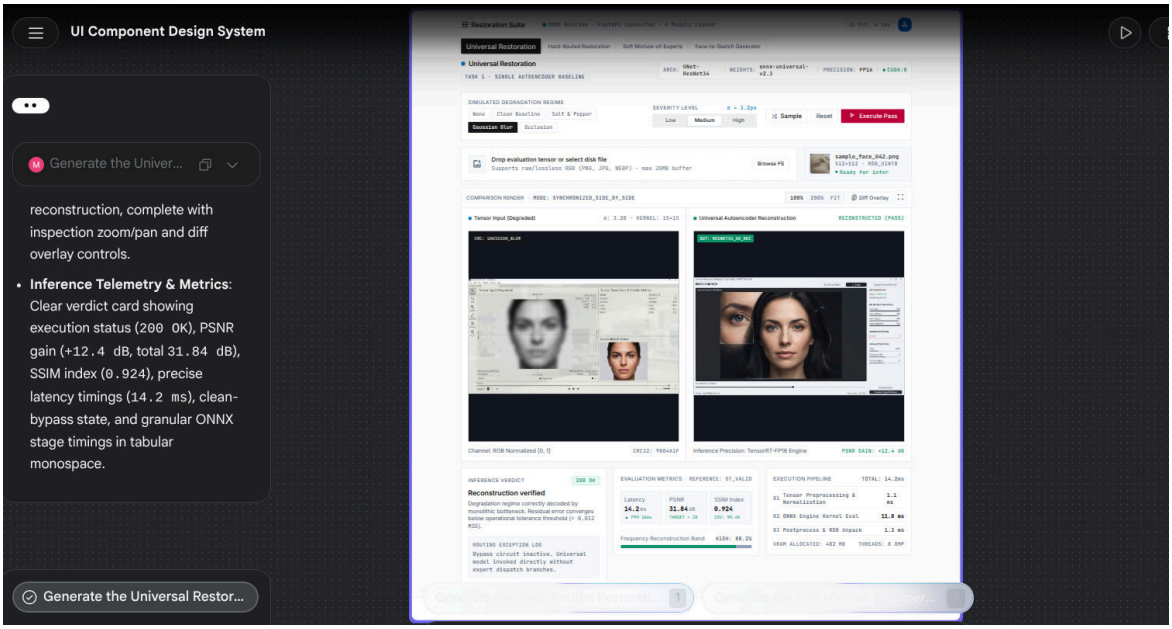


Fig. 27. Original Google Stitch design: prompt history (left) and generated *Universal Restoration* workspace with the four-tab structure (right).

configuration is required to report the trained model's test performance. Future work: identity-preserving skip gating and longer cosine schedules.

Task 2. A balanced-batch classifier (98.3% accuracy) routes inputs to specialists that restore far better than the evaluated universal model. Its errors are concentrated on mild corruptions misread as clean. Future work: confidence-thresholded fallback to a soft combination.

Task 3. A soft MoE initialised from Task 2 and trained with a batch-level balance loss improves on hard routing for every corrupted condition, without inactive or dominating experts. It costs three expert passes per image. Future work: top-k sparse evaluation and mixed-corruption training.

Task 4. The style-conditioned pix2pix generator produces distinguishable styles (83% style match) without mode collapse, but loses hair detail. A filename-collision pairing error was discovered in failure analysis. Future work: corrected pairing, perceptual or LPIPS losses and higher resolution.

Appendix A
AI-Use Disclosure

Claude (Anthropic), through Claude Code, was used for:

- reviewing the notebooks against the rubric and adding cells that run on the saved checkpoints, for sample and severity grids, Optuna plots, training curves, failure cases with error maps, ONNX-vs-PyTorch checks, style-confusion and mode-collapse analysis, and MLflow logging;
- back-end fixes: upload validation and the Task 4 $[-1, 1]$ scaling;
- adding Tailwind CSS;
- drafting this report and the README.

The outputs were verified by executing every notebook end to end on Colab and checking that each reported number appears in the cell outputs. The generated code was checked for undefined names before running. The fixed Task 4 scaling was compared against the raw ONNX output range, and the frontend was confirmed to compile (`vite build`). All design decisions, training runs and results are the author's. Reused material: the FS2K and Oxford-IIIT Pet datasets, the PyTorch, `pytorch-msssim`, Optuna, MLflow and ONNX Runtime libraries, and the ideas adapted from AirNet [3] and pix2pix [5], as cited.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 2623–2631.

[2] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, pp. 3371–3408, 2010.

[3] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv, and X. Peng, "All-in-one image restoration for unknown corruption," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 17452–17462.

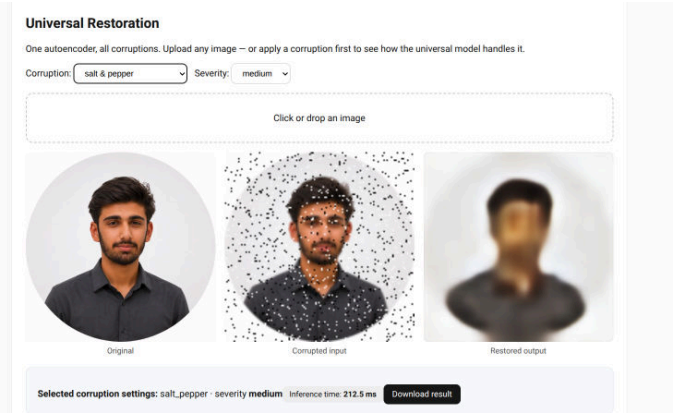
[4] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[5] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1125–1134.

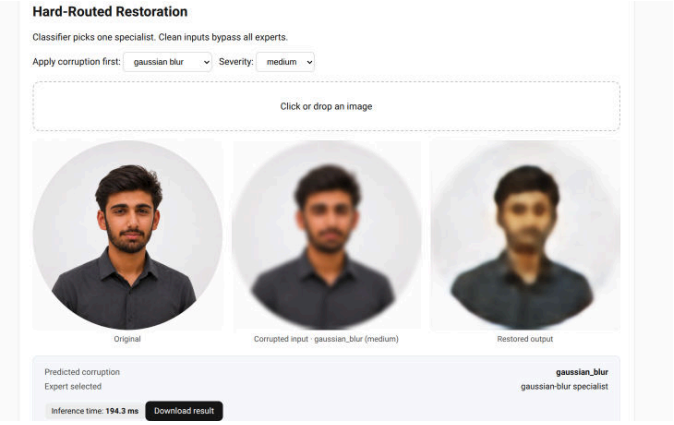
[6] A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016.

[7] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 1597–1607.

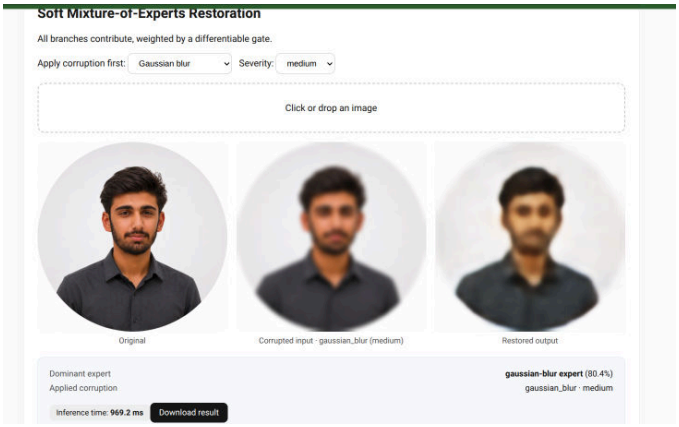
[8] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, 2018.



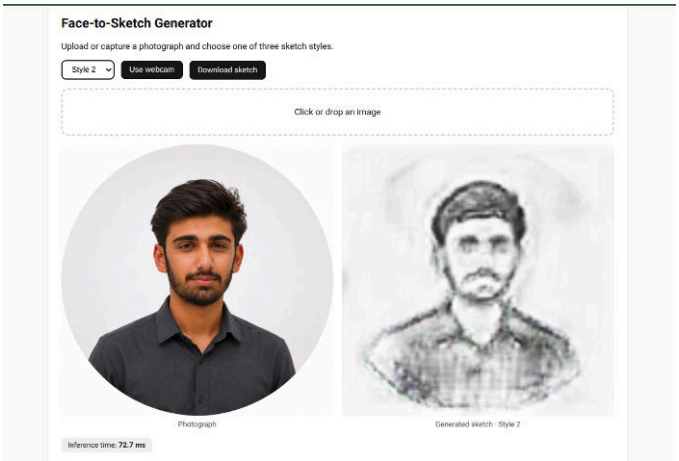
(a) Universal Restoration (salt-and-pepper, medium).



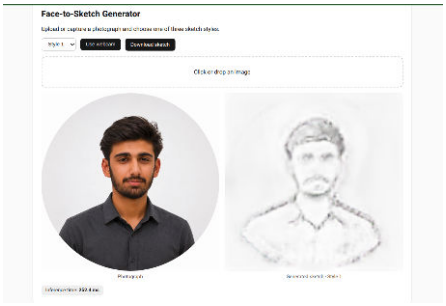
(b) Hard-Routed Restoration (blur, medium).



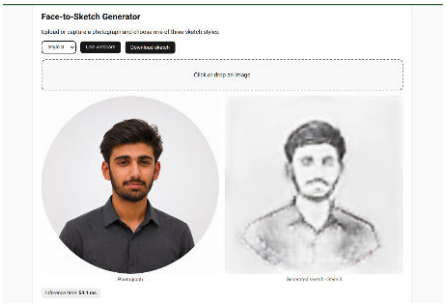
(c) Soft Mixture-of-Experts Restoration.



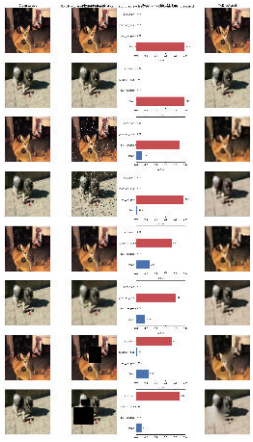
(d) Face-to-Sketch Generator, Style 2.



(e) Style 1.



(f) Style 3.



(g) Soft-MoE preview from the notebook.

Fig. 28. Application screenshots of the four workspaces on a previously unseen photograph (input, corruption controls, outputs, predicted corruption/selected expert or dominant expert, inference time and download). The hard-routed and soft-MoE workspaces also render the four class probabilities and the four routing weights as bars below the result card; the same information is shown for test images in Fig. 13 and panel (g).

[9] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2536–2544.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[11] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Trans. Comput. Imaging*, vol. 3,

no. 1, pp. 47–57, 2017.
[12] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 694–711.
[13] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.
[14] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
[15] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Comput.*, vol. 3, no. 1, pp. 79–87, 1991.
[16] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, "From sparse to soft mixtures of experts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
[17] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 9729–9738.
[18] I. Goodfellow *et al.*, "Generative adversarial nets," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2014, pp. 2672–2680.
[19] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv:1411.1784, 2014.
[20] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," arXiv:1607.08022, 2016.
[21] Y. Wu and K. He, "Group normalization," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3–19.
[22] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2011, pp. 2546–2554.
[23] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.
[24] ONNX Runtime developers, "ONNX Runtime," 2021. [Online]. Available: https://onnxruntime.ai
[25] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3498–3505.
[26] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. Van Gool, "Facial-sketch synthesis: A new challenge," *Mach. Intell. Res.*, vol. 19, no. 4, pp. 257–287, 2022.